

Experiment No.	8
Experiment Title	Clustering Human Activity Recognition Data using K-Means, DBSCAN and Hierarchical Clustering
Student Name	Hemnath D
Register Number	3122247001022
Course	M.Tech Integrated Computer Science and Engineering
Faculty	Dr. Poreddy Ajay Kumar Reddy
Date of Submission	19 September 2026

1 Objective

The aim of this experiment is to cluster the UCI Human Activity Recognition (HAR) smartphone dataset with three unsupervised algorithms, K-Means (Model A), DBSCAN (Model B) and Hierarchical Agglomerative Clustering (Model C), and to compare the clusters they produce. The number of K-Means clusters is chosen with the elbow and silhouette methods, DBSCAN's ϵ and $minPts$ are tuned with a k-distance plot and a grid search, and the effect of the linkage criterion on hierarchical clustering is studied. Each solution is scored with internal metrics (Silhouette, Davies–Bouldin, Calinski–Harabasz) and, because the true activity labels are available, with external metrics (ARI, NMI) and mapped confusion matrices.

2 Problem Statement

Each row of the HAR dataset is a 2.56 s window of accelerometer and gyroscope readings from a smartphone worn on the waist, summarised into 561 time and frequency domain features. The six activities (walking, walking upstairs, walking downstairs, sitting, standing, laying) are known, but in this experiment the labels are hidden from the algorithms and only used afterwards to judge the clusters. The question is whether the natural geometric structure of the sensor features matches the six activities, how each algorithm's assumptions (spherical clusters for K-Means, density for DBSCAN, variance-minimising merges for Ward) shape the result, and which internal metric is a reliable guide when no labels are available.

3 Dataset Description

Dataset Name	Human Activity Recognition Using Smartphones
Dataset Source	UCI Machine Learning Repository (Anguita et al., 2013)
Number of Samples	10,299 windows (7,352 train + 2,947 test, merged) from 30 volunteers aged 19–48
Number of Features	561 (means, standard deviations, energies, entropies, correlations and FFT coefficients of body/gravity acceleration and gyroscope signals), all already scaled to $[-1, 1]$
Number of Classes	6 activities: WALKING (1,722), WALKING_UPSTAIRS (1,544), WALKING_DOWNSTAIRS (1,406), SITTING (1,777), STANDING (1,906), LAYING (1,944)
Missing Values	None; no duplicate rows

Train-Test Split	Not used. Clustering is unsupervised, so the provided train and test partitions were merged and all 10,299 windows were clustered together
------------------	--

4 Exploratory Data Analysis

The dataset has no missing values, no duplicates, and every feature already lies in $[-1, 1]$. The 561 features come from 17 signal groups (for example 79 each from the FFT of body acceleration, gyroscope and jerk signals, and 40 each from the time domain body and gravity signals). Activities are fairly balanced, between 13.7% and 18.9% of the windows each, and every subject contributes between 281 and 409 windows.

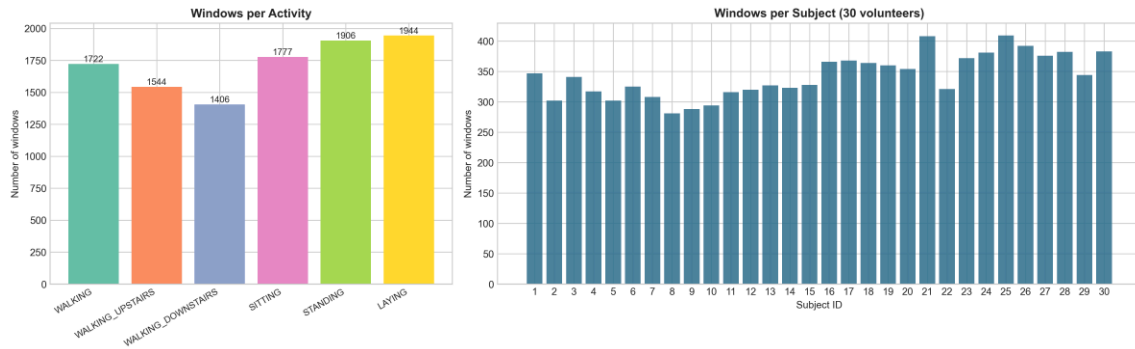


Figure 1: Number of windows per activity (left) and per subject (right).

Inference: The six activities are close to balanced, so the ground truth is six clusters of roughly 1,400 to 1,950 windows. This matters for the comparison later: an algorithm that produces one huge cluster and a few tiny ones is clearly not recovering the activities. The per-subject counts are also even, so no volunteer dominates the data.

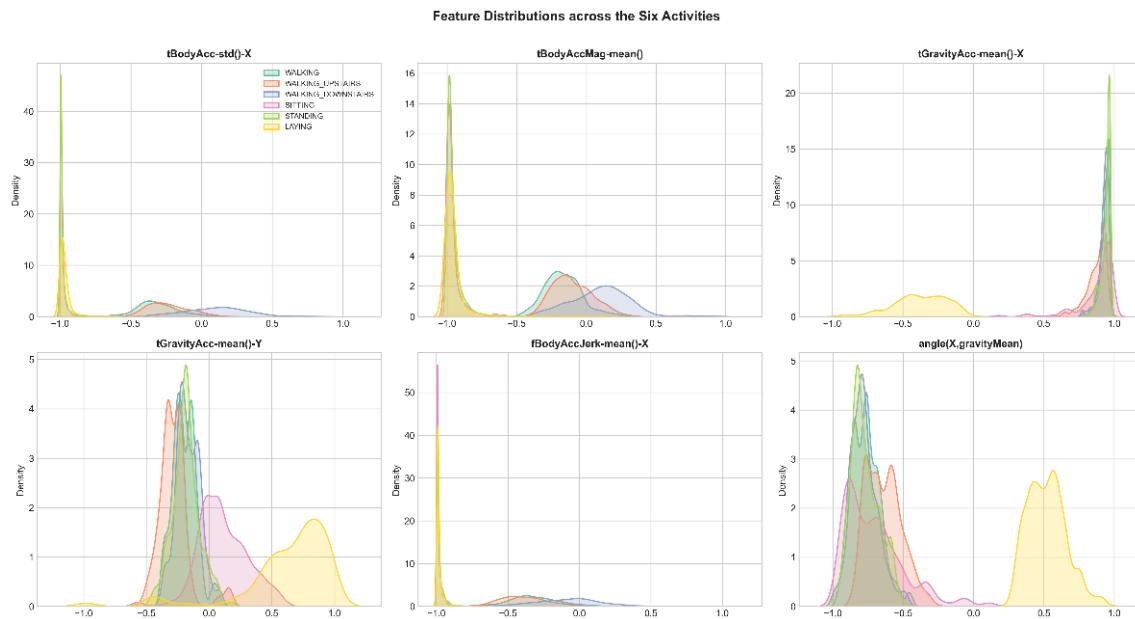


Figure 2: Distribution of six representative features for each activity.

Inference: The body acceleration standard deviation and magnitude features split the

data into two very clean groups: the three static activities pile up at -1 while the walking activities spread over higher values. Gravity related features separate LAYING from SITTING and STANDING because the phone's orientation changes when the person lies down. Within the walking group and within SITTING/STANDING the distributions overlap heavily, which already suggests that the strongest structure in the data is a two-way static/dynamic split rather than six separate groups.

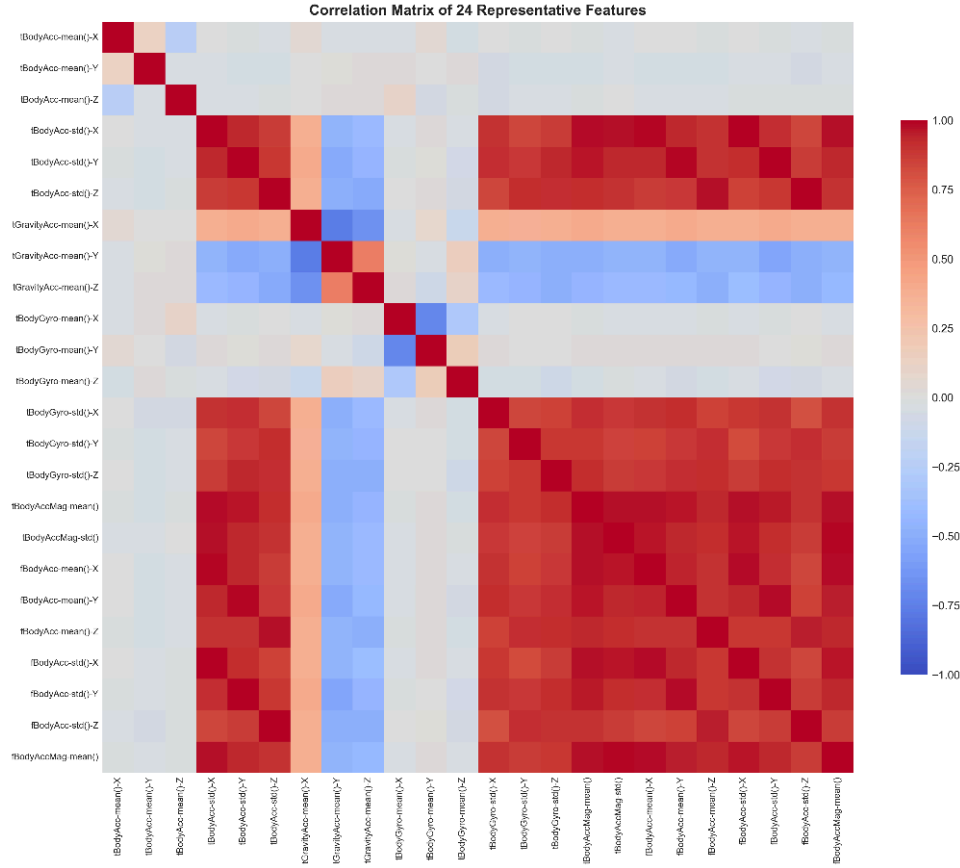


Figure 3: Correlation matrix of 24 representative mean and standard deviation features.

Inference: The features are extremely redundant. Across all 157,080 feature pairs, 5.2% have $|r| > 0.9$ and 14.5% have $|r| > 0.8$, with a mean $|r|$ of 0.42. Standard deviation, magnitude and frequency-domain features of the same signal are near duplicates. This redundancy is the reason PCA is used before clustering: distances computed on 561 correlated features count the same information many times over.

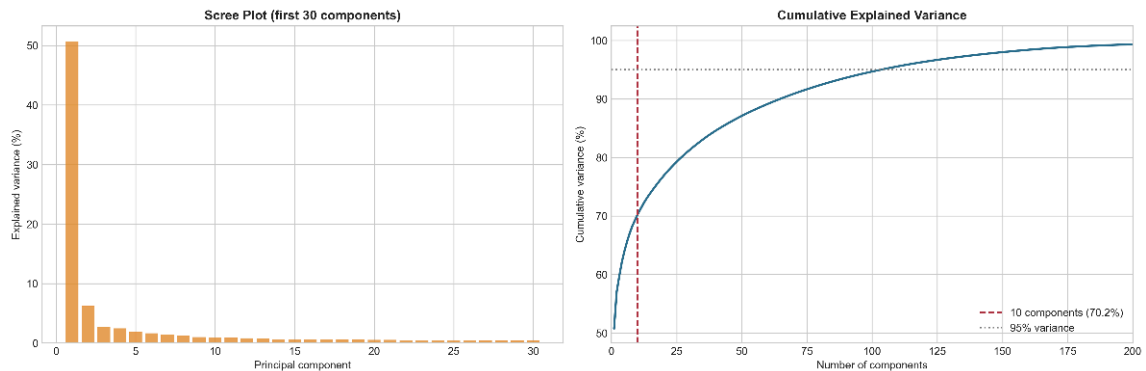


Figure 4: Scree plot of the first 30 principal components and the cumulative explained variance.

Inference: The first two components already explain 57.0% of the variance and ten components explain 70.2%, while 104 components are needed to reach 95%. The variance is therefore concentrated in a handful of directions followed by a long, flat tail of low-variance components that mostly carry noise. Ten components were chosen as the common working space for all three algorithms.

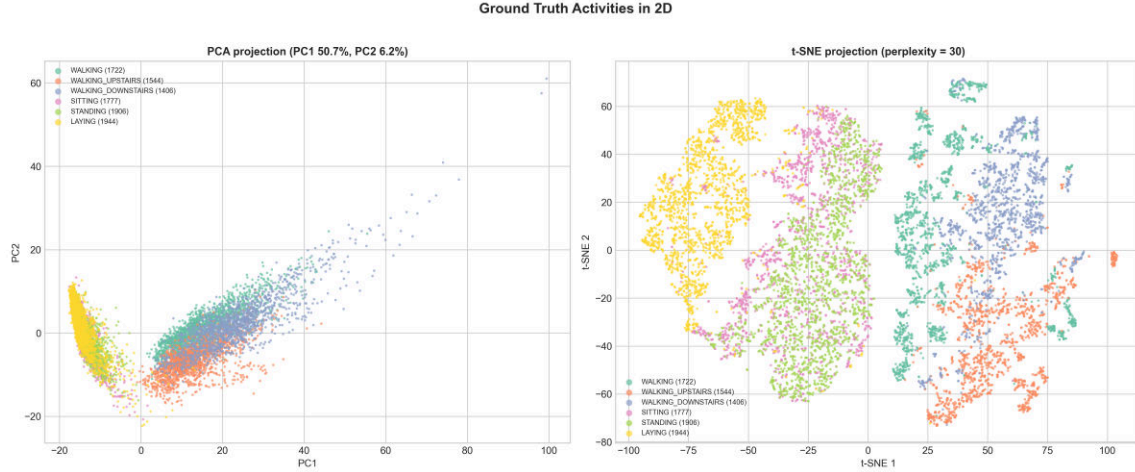


Figure 5: PCA (left) and t-SNE (right) projections coloured by the true activity.

Inference: Both projections show two well separated blobs, the static activities on one side and the walking activities on the other. LAYING forms its own region inside the static blob, but SITTING and STANDING are mixed, and the three walking variants form overlapping bands rather than isolated groups. This is the visual reference for judging the algorithms: a good unsupervised result should find the two big groups and LAYING, and it is unrealistic to expect a clean separation of SITTING from STANDING or of the walking variants.

5 Data Preprocessing

- **Missing value handling:** No missing values were present, so no imputation was needed. Duplicate feature names in the original `features.txt` were made unique with a numeric suffix.
- **Encoding:** Activity labels were already integer coded (1–6); the label names were attached only for reporting. Labels were never given to the clustering algorithms.
- **Feature scaling:** Although the features lie in $[-1, 1]$, their spreads differ a lot (for example `tBodyAcc-mean()-X` has a standard deviation of 0.07 and `tGravityAcc-mean()-X` of 0.52). `StandardScaler` was applied so that every feature contributes equally to Euclidean distances.
- **Dimensionality reduction:** PCA with 10 components (70.2% of the variance) was fitted on the standardized data and used as the common feature space for K-Means, DBSCAN and hierarchical clustering. This removes the redundancy seen in the correlation analysis, gives DBSCAN a space in which density is meaningful (in 561 dimensions all neighbour distances are similar and DBSCAN either merges everything or marks a third of the data as noise), and makes the $O(n^2)$ hierarchical clustering fast. A check with K-Means confirmed that the 10-D space does not hurt: ARI was 0.420 in the full space, 0.420 with 50 components and 0.427 with 10 components.
- **Visualization:** A 2-D PCA projection and a t-SNE embedding (perplexity 30, PCA initialisation) of the 10-D space were computed once and reused for all cluster plots.

- **Train-test split:** Not applicable; all 10,299 windows were clustered.

6 Machine Learning Algorithms

6.1 K-Means

K-Means picks k centroids, assigns every point to the nearest one, moves each centroid to the mean of its points and repeats until nothing changes. What it minimises is the within-cluster sum of squares

$$J = \sum_{j=1}^k \sum_{\mathbf{x}_i \in C_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2.$$

It is fast and easy to read, but k has to be given, it assumes round clusters of similar size, it can land in a poor local minimum depending on the starting centroids (so 10 restarts were used), and every point gets a cluster whether it fits or not.

6.2 DBSCAN

DBSCAN looks at density instead of centroids. A point with at least *minPts* neighbours within radius ϵ is a core point; core points that are within ϵ of each other join the same cluster, points near a cluster but without enough neighbours are border points, and everything else is noise. This lets it find clusters of any shape and flag outliers without being told how many clusters there are. The catch is that ϵ and *minPts* are awkward to set, one ϵ cannot suit clusters of very different density, and in high dimensions all points end up roughly equally far apart so there is no density contrast left to work with.

6.3 Hierarchical Agglomerative Clustering

Agglomerative clustering starts with every point as its own cluster and keeps merging the two closest clusters until one is left, which gives a dendrogram that can be cut at any number of clusters. How “closest” is measured is the linkage: single uses the nearest pair of points, complete the farthest pair, average the mean distance, and Ward merges whichever pair increases the total within-cluster variance the least,

$$\Delta(A, B) = \frac{|A||B|}{|A| + |B|} \|\boldsymbol{\mu}_A - \boldsymbol{\mu}_B\|^2.$$

The dendrogram is a nice picture of the structure at every scale and the result is deterministic, but memory and time grow with n^2 , merges cannot be undone, and single linkage in particular tends to chain everything together.

6.4 Evaluation Metrics

Silhouette compares how close a point is to its own cluster against the nearest other cluster (higher is better). Davies–Bouldin averages the worst ratio of cluster spread to centroid distance (lower is better). Calinski–Harabasz is the ratio of between-cluster to within-cluster dispersion (higher is better). ARI and NMI compare the clusters with the true activities and are 1 for a perfect match and about 0 for random labels. Internal metrics were computed in the 10-D PCA space on non-noise points, and clusters were matched to activities with the Hungarian algorithm to draw confusion matrices.

7 Experimental Setup

Python Version	3.13.5
Key Libraries	scikit-learn 1.7.2, SciPy 1.16.2, NumPy, pandas, Matplotlib, Seaborn
IDE	Jupyter Notebook / VS Code
Operating System	macOS 26.6.2
Hardware	Apple M3 Pro, 18 GB RAM (CPU only)

8 Hyperparameter Selection

8.1 K-Means: elbow and silhouette methods

K-Means was run for $k = 2$ to 8 with 10 random restarts each. Table 2 is the elbow table required by the experiment sheet, extended with the other metrics.

Table 2: K-Means elbow method results (10-D PCA space, `n_init=10`)

k	WCSS (Inertia)	WCSS drop	Silhouette	Davies–Bouldin	Calinski–Harabasz	ARI	NMI
2	1,552,171	—	0.5606	0.694	16,613	0.3296	0.5455
3	1,204,169	22.4%	0.4734	1.084	12,194	0.3314	0.5188
4	1,067,331	11.4%	0.2967	1.373	9,611	0.2988	0.4698
5	942,608	11.7%	0.2727	1.400	8,501	0.2909	0.4603
6	866,550	8.1%	0.2806	1.287	7,578	0.4271	0.5621
7	815,451	5.9%	0.2733	1.244	6,818	0.4140	0.5564
8	761,871	6.6%	0.2480	1.375	6,357	0.4101	0.5438

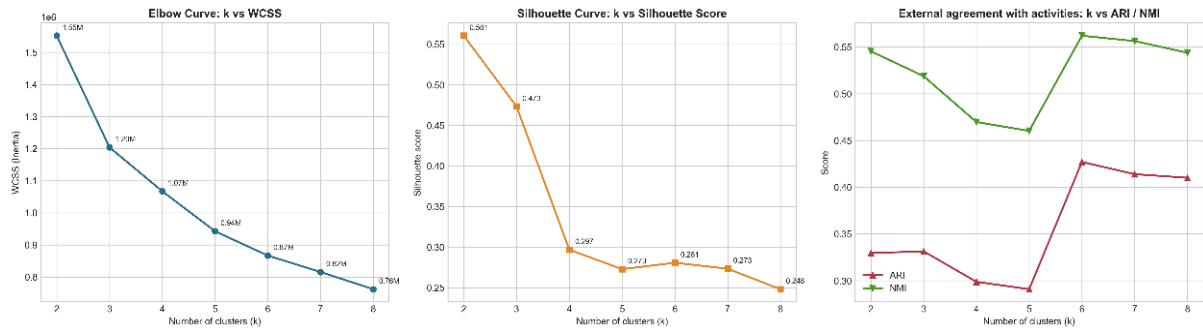


Figure 6: Elbow curve (k vs WCSS), silhouette curve (k vs silhouette) and external agreement (k vs ARI/NMI).

Inference: The WCSS curve falls steeply up to $k = 3$ (a 22.4% drop from $k = 2$ to 3) and flattens after $k = 6$, where the per-step reduction drops to about 6%. The silhouette is highest at $k = 2$ (0.561), which is the static/dynamic split, and then falls sharply; among the finer solutions it has a small local peak at $k = 6$ (0.281 against 0.273 at $k = 5$ and 7). ARI and NMI jump at $k = 6$ (0.427 and 0.562) and decline afterwards.

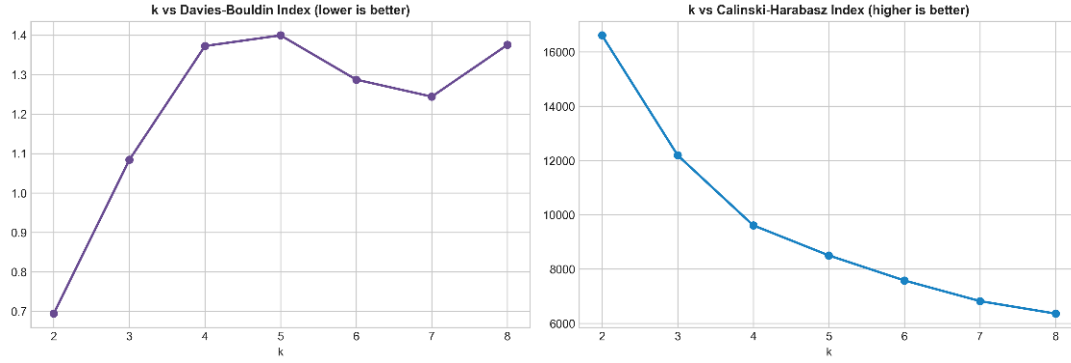


Figure 7: Davies–Bouldin and Calinski–Harabasz indices for $k = 2$ to 8.

Inference: Both indices also favour $k = 2$, and Davies–Bouldin has a secondary dip at $k = 6$ to 7. All internal metrics therefore agree that the two-group structure dominates, but the elbow region, the local silhouette dip and the number of activities all point to six clusters for the fine-grained solution. $k = 6$ **was selected** for the final K-Means model and the $k = 2$ solution was kept for comparison.

8.2 DBSCAN: ϵ and $minPts$

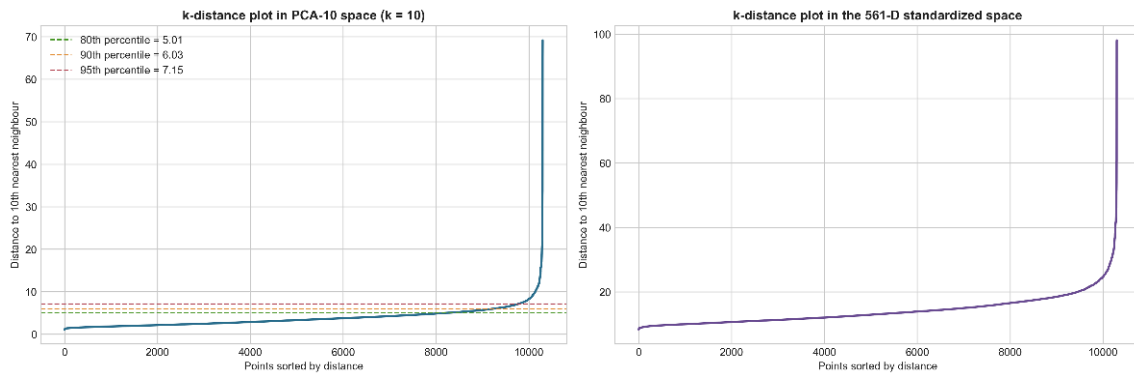


Figure 8: Sorted 10-nearest-neighbour distance in the 10-D PCA space (left) and in the full 561-D space (right).

Inference: In the PCA space the k-distance curve is flat for most points and then rises sharply, with the knee between the 80th and 95th percentiles (5.0 to 7.2), so ϵ was searched in the range 2.5 to 7.0. In the 561-D space the ratio between the largest and smallest k-distance is only 11.7 (versus 61.8 in PCA space) and the curve has no knee; DBSCAN there either merged everything into one cluster (5.7% noise at $\epsilon = 19.3$) or produced 12 fragments with 38.5% noise, which is why DBSCAN was run in the PCA space.

Table 3: DBSCAN grid search (silhouette computed on non-noise points). Selected setting in bold.

ϵ	$minPts$	Clusters	Noise (%)	Silhouette	Davies–Bouldin	Calinski–Harabasz	ARI	NMI
2.5	5	53	52.6	−0.181	1.073	106	0.186	0.331
2.5	10	12	59.9	−0.063	1.023	126	0.152	0.286
2.5	20	1	66.3	—	—	—	0.111	0.234
3.0	5	45	38.6	−0.013	1.186	315	0.240	0.388
3.0	10	19	47.9	0.514	1.114	383	0.218	0.373
3.0	20	4	55.4	0.571	0.770	468	0.195	0.332
3.5	5	30	27.4	0.210	0.961	619	0.293	0.441
3.5	10	10	34.9	0.443	1.024	1,485	0.266	0.413
3.5	20	9	44.5	0.521	1.180	985	0.239	0.402
4.0	5	30	18.1	0.165	0.929	708	0.308	0.456
4.0	10	8	24.9	0.431	1.191	2,379	0.307	0.456
4.0	20	5	33.3	0.484	1.044	3,327	0.276	0.423
4.5	5	18	11.2	0.269	0.810	1,283	0.317	0.482
4.5	10	9	16.8	0.380	0.948	2,498	0.314	0.468
4.5	20	3	23.6	0.516	0.727	8,310	0.313	0.464
5.0	5	13	7.2	0.404	0.854	1,796	0.322	0.499
5.0	10	6	10.8	0.447	0.723	4,193	0.317	0.485
5.0	20	3	16.9	0.474	0.812	9,432	0.313	0.470
5.5	5	10	4.7	0.427	0.727	2,350	0.325	0.514
5.5	10	7	7.1	0.483	0.791	3,599	0.323	0.504
5.5	20	4	11.6	0.450	0.823	6,875	0.314	0.481
6.0	5	7	3.2	0.418	0.770	3,450	0.327	0.525
6.0	10	4	5.0	0.533	0.616	7,017	0.325	0.515
6.0	20	3	8.1	0.561	0.622	10,399	0.321	0.500
6.5	5	2	2.3	0.355	0.551	14	0.003	0.015
6.5	10	3	3.4	0.304	0.648	108	0.005	0.032
6.5	20	3	5.9	0.553	0.639	10,502	0.324	0.512
7.0	5	2	1.9	0.477	0.451	33	0.002	0.014
7.0	10	2	2.6	0.318	0.632	43	0.003	0.019
7.0	20	2	4.1	0.371	0.648	185	0.006	0.036

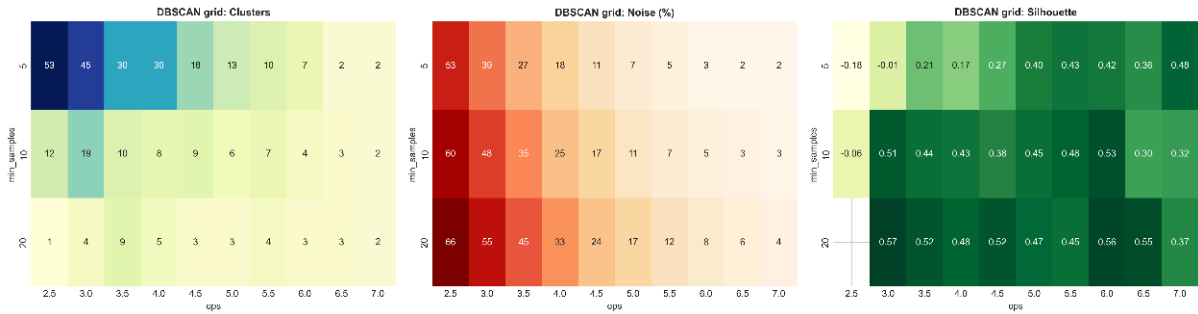


Figure 9: Number of clusters, noise fraction and silhouette over the DBSCAN grid.

Inference: Small ϵ shatters the data into dozens of fragments with more than half the windows as noise; large ϵ (≥ 6.5 with small $minPts$) bridges the gap between the static and dynamic groups and collapses everything into one cluster plus a tiny satellite (ARI drops to about 0). The usable band is $\epsilon = 5$ to 6.5. To keep the choice label-free, the setting with the highest silhouette among those with less than 10% noise and at least three clusters was selected: $\epsilon = 6.0$, $minPts = 20$ (3 clusters, 8.1% noise, silhouette 0.561). A finer alternative ($\epsilon = 5.5$, $minPts = 10$, 7 clusters, 7.1% noise) was also kept to see whether DBSCAN could split the groups further.

8.3 Hierarchical clustering: linkage criterion

Table 4: Agglomerative clustering with different linkage criteria (6 clusters, 10-D PCA space)

Linkage	Cluster sizes	Silhouette	DB	CH	ARI	NMI	Time (s)
Ward	3344 / 2399 / 2113 / 1717 / 565 / 161	0.263	1.305	6,713	0.490	0.610	1.11
Complete	9912 / 181 / 101 / 97 / 6 / 2	0.208	1.059	379	0.004	0.042	1.10
Average	5573 / 4545 / 108 / 50 / 21 / 2	0.533	0.840	4,433	0.330	0.543	1.21
Single	10293 / 2 / 1 / 1 / 1 / 1	0.370	0.311	32	0.000	0.001	0.20

Ward’s linkage was selected, as required by the experiment sheet and as the only criterion that produces balanced clusters. The merge heights of the Ward dendrogram show the natural cuts: the final merge (2 clusters) happens at height 2,236, far above the next at 754 (3 clusters), with smaller gaps at 6 clusters (427 to 360). The tree was cut at six clusters (height 393.5) to match the activities.

Table 5: Final hyperparameters

Parameter	Value
Feature space (all models)	StandardScaler + PCA, 10 components (70.2% variance)
K-Means	$k = 6$, k-means++ initialisation, <code>n_init = 10</code> , <code>random_state = 42</code>
DBSCAN	$\epsilon = 6.0$, $minPts = 20$, Euclidean distance (alternative: $\epsilon = 5.5$, $minPts = 10$)
Hierarchical	Agglomerative, Ward linkage, cut at 6 clusters
t-SNE (visualisation only)	2 components, perplexity 30, PCA initialisation

9 Results

Table 6: Evaluation metrics of all clustering solutions (internal metrics in the 10-D PCA space)

Algorithm	Clusters	Noise	Silhouette	DB	CH	ARI	NMI	Mapped Acc.
K-Means ($k = 6$)	6	0%	0.2806	1.287	7,578	0.4271	0.5621	54.6%
K-Means ($k = 2$)	2	0%	0.5606	0.694	16,613	0.3296	0.5455	35.5%
DBSCAN ($\epsilon = 6.0$, $minPts = 20$)	3	8.1%	0.5614	0.622	10,399	0.3210	0.4996	36.4%
DBSCAN alt ($\epsilon = 5.5$, $minPts = 10$)	7	7.1%	0.4834	0.791	3,599	0.3229	0.5035	36.7%
Hierarchical Ward ($k = 6$)	6	0%	0.2626	1.305	6,713	0.4897	0.6096	59.6%
Hierarchical Complete ($k = 6$)	6	0%	0.2076	1.059	379	0.0036	0.0416	20.4%

Table 7: Contingency tables of the three final models against the true activities

Model	Cluster	WALK	UP	DOWN	SIT	STAND	LAY
K-Means	0	0	0	0	118	0	1568
	1	896	1236	319	1	0	3
	2	749	300	889	0	0	0
	3	0	2	0	391	445	316
	4	0	0	0	1267	1461	57
	5	77	6	198	0	0	0
DBSCAN	noise	199	110	414	34	16	57
	0	0	0	0	1743	1890	1887
	1	1523	1400	992	0	0	0
	2	0	34	0	0	0	0
Ward	0	0	0	0	76	0	1641
	1	0	0	0	243	109	213
	2	0	0	0	1457	1797	90
	3	628	1259	225	1	0	0
	4	22	13	126	0	0	0
	5	1072	272	1055	0	0	0

Table 8: Per-activity recall (%) after mapping clusters to activities with the Hungarian algorithm

Activity	K-Means	DBSCAN	Ward
WALKING	4.5	100.0	62.3
WALKING_UPSTAIRS	80.1	2.4	81.5
WALKING_DOWNSTAIRS	63.2	0.0	9.0
SITTING	22.0	0.0	13.7
STANDING	76.7	100.0	94.3
LAYING	80.7	0.0	84.4

Ward linkage gives the best agreement with the activities (ARI 0.490, NMI 0.610, 59.6% mapped accuracy), K-Means with $k = 6$ is second (ARI 0.427, NMI 0.562, 54.6%), and DBSCAN, although it has the best internal scores, only recovers the static/dynamic split plus one small cluster (ARI 0.321, NMI 0.500). The DBSCAN recall column looks extreme because with only three clusters the Hungarian mapping can assign each big cluster to a single activity, so WALKING and STANDING get 100% and the activities merged with them get 0%.

10 Result Visualization

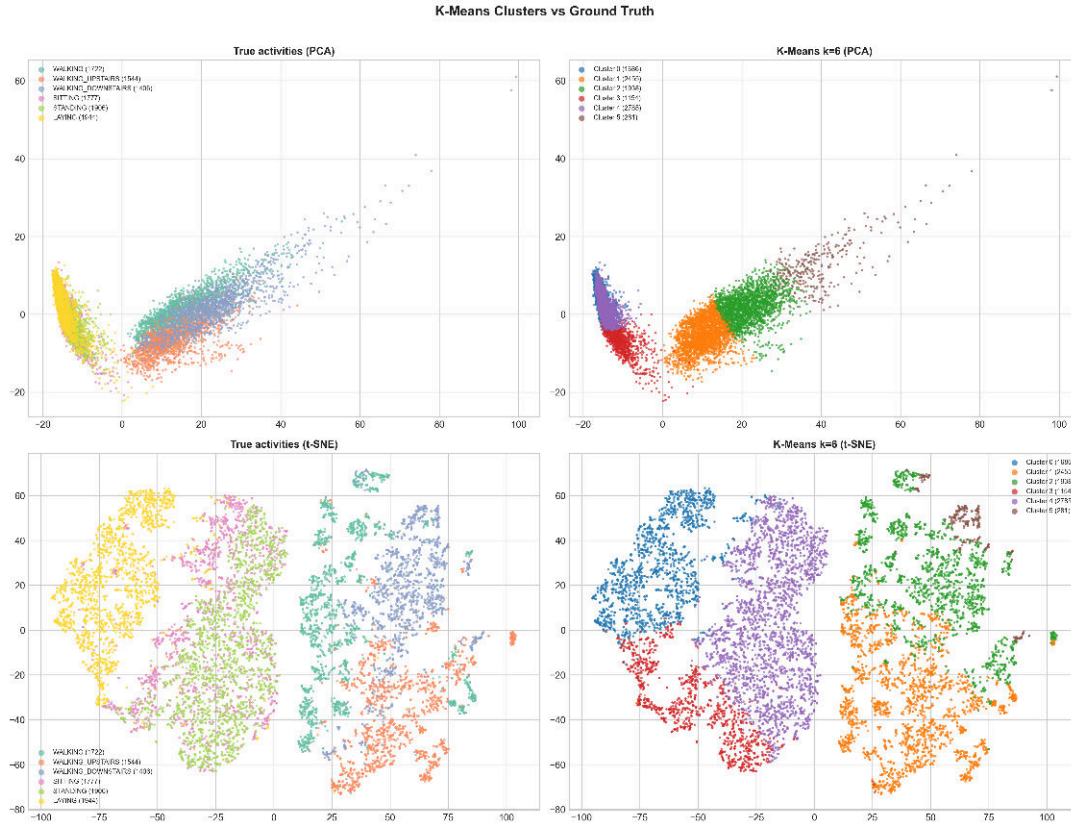


Figure 10: K-Means ($k = 6$) clusters next to the ground truth in the PCA and t-SNE projections.

Inference: K-Means splits the static blob into a LAYING cluster (cluster 0, 1,568 LAYING windows), a large SITTING/STANDING cluster (cluster 4) and a mixed cluster (cluster 3), and splits the dynamic blob into an upstairs-heavy cluster (cluster 1, 1,236 of the 1,544 upstairs windows), a walking/downstairs cluster (cluster 2) and a small downstairs cluster (cluster 5, 281 windows). The cuts are straight and roughly equal-sized, as expected from an algorithm that minimises variance around centroids, and they slice through the SITTING/STANDING band rather than following it.

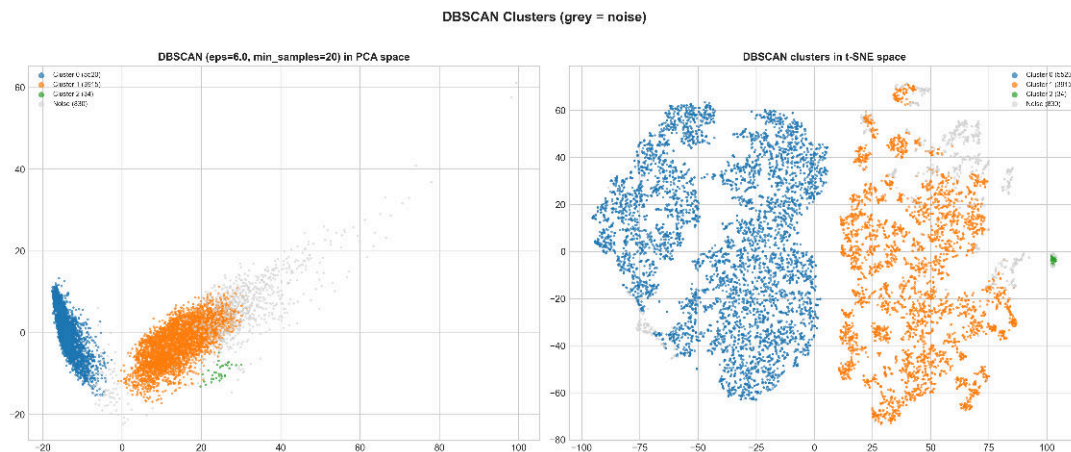


Figure 11: DBSCAN ($\epsilon = 6.0$, $minPts = 20$) clusters; grey points are noise.

Inference: DBSCAN finds exactly the two dense regions, 5,520 static and 3,915 dynamic windows, plus a cluster of 34 WALKING_UPSTAIRS windows that sits apart from the rest. The 830 noise points (grey) lie at the fringes of the dynamic group and are mostly WALKING_DOWNSTAIRS (414) and WALKING (199) windows, which are the most variable movements. Because SITTING/STANDING and the walking variants form continuous regions of similar density, there is no density gap for DBSCAN to split them at.

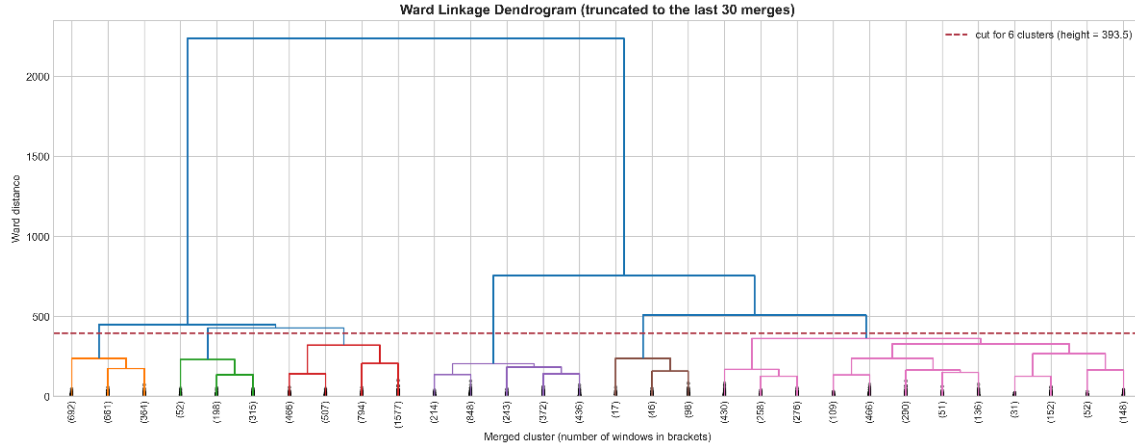


Figure 12: Ward linkage dendrogram of all 10,299 windows, truncated to the last 30 merges, with the six-cluster cut.

Inference: The dendrogram has one very tall final merge (height 2,236) joining the static and dynamic branches, a second clear merge at 754 inside the static branch that separates LAYING from SITTING/STANDING, and then a set of merges between 360 and 510 that produce the six-cluster solution. The heights make the hierarchy of the data explicit: two strong groups, three clear groups, and six plausible but much weaker groups.

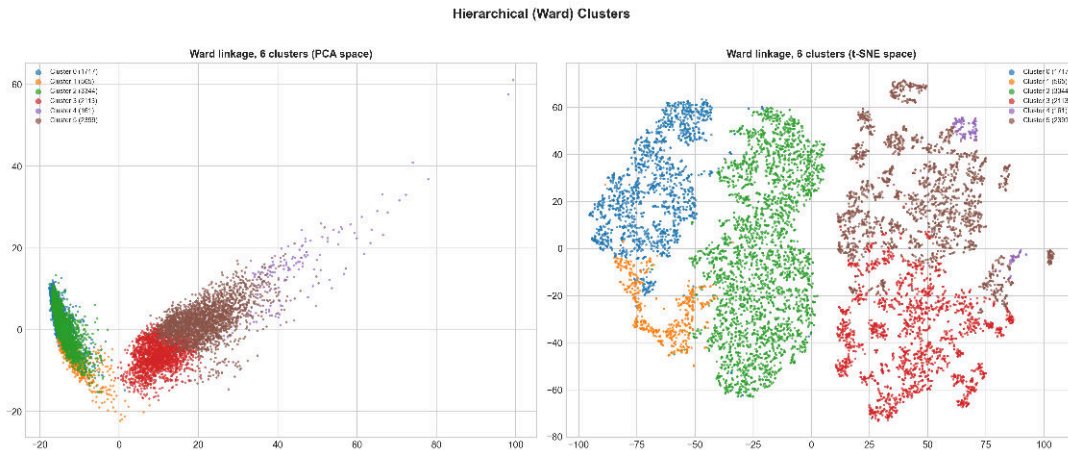


Figure 13: Ward clusters (6) in the PCA and t-SNE projections.

Inference: Ward's clusters follow the shape of the t-SNE blobs better than K-Means. LAYING is almost isolated (1,641 of 1,944 windows in cluster 0), SITTING and STANDING share one large cluster of 3,254 windows, and the walking windows are split into an upstairs cluster (1,259 of 1,544) and a walking/downstairs cluster, with a small cluster of 161 downstairs windows. This is why Ward reaches the highest ARI and NMI.



Figure 14: Six-cluster solutions with Ward, complete, average and single linkage in the t-SNE view.

Inference: Only Ward produces a sensible partition. Single linkage chains every point into one cluster and leaves five singletons, average linkage recovers just the static/dynamic split with four tiny satellites, and complete linkage puts 9,912 windows in one cluster and scatters the rest into small groups of outliers. Linkages that are driven by extreme pairwise distances let a few outlying windows form their own clusters and lump everything else together.

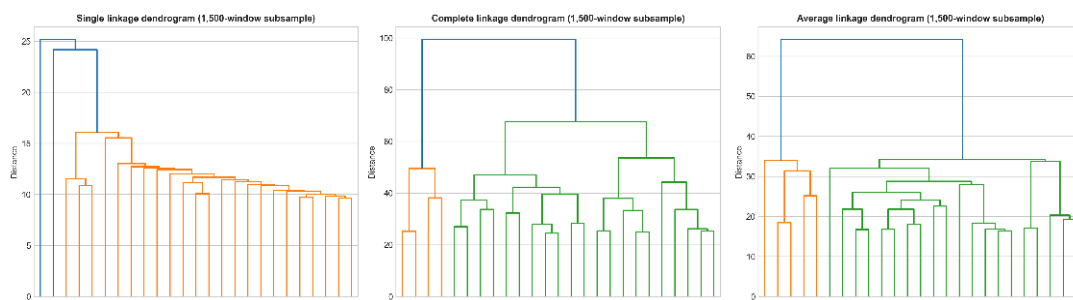


Figure 15: Dendrograms for single, complete and average linkage on a stratified 1,500-window subsample.

Inference: The single-linkage dendrogram has no tall branches at all, just a staircase of tiny merges, which is the signature of chaining. Complete and average linkage do show a tall final merge but their intermediate structure is dominated by a few outlying points that merge last, so cutting them at six clusters yields outlier clusters instead of activity clusters.

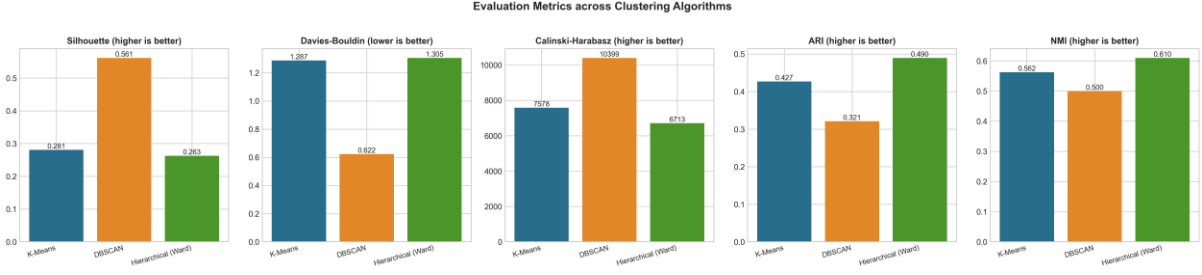


Figure 16: Internal and external metrics of the three final models.

Inference: The internal and external metrics disagree. DBSCAN wins all three internal metrics (silhouette 0.561, Davies–Bouldin 0.622, Calinski–Harabasz 10,399) because its three clusters coincide with the two big well separated blobs, while Ward wins both external metrics (ARI 0.490, NMI 0.610) because it actually splits those blobs into activity-like groups, at the cost of lower compactness scores. Internal metrics measure geometry, and the geometry of this dataset has two groups, not six.

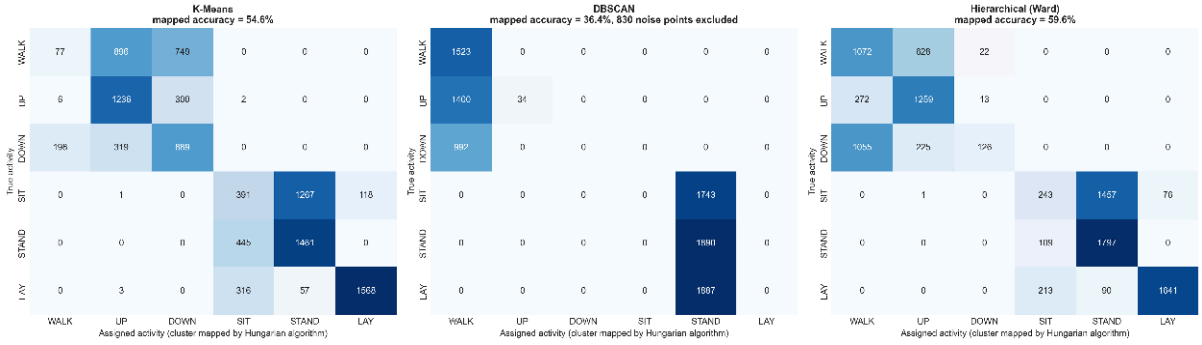


Figure 17: Confusion matrices after mapping clusters to activities with the Hungarian algorithm.

Inference: All three matrices have the same block structure: no window is ever confused across the static/dynamic boundary. Within the blocks, SITTING is mostly absorbed into STANDING and WALKING_DOWNSTAIRS into WALKING, for every algorithm. The DBSCAN matrix has only three occupied columns because it only has three clusters, so the mapping can serve just three activities.

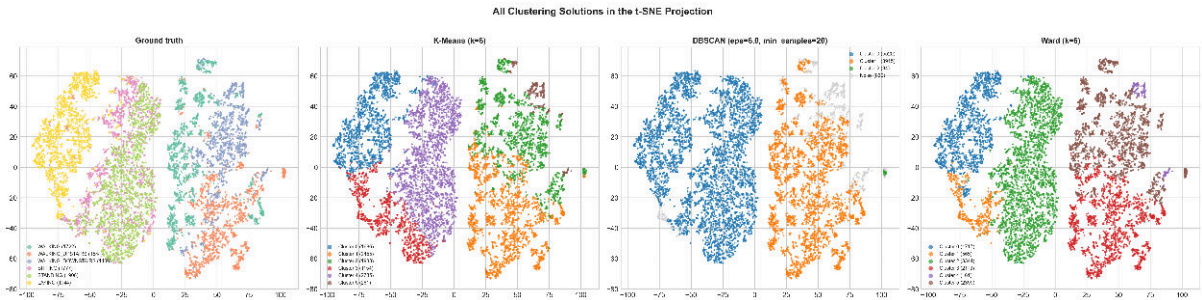


Figure 18: Ground truth and all three clustering solutions in the t-SNE projection.

Inference: Seen side by side, DBSCAN reproduces the coarse layout, K-Means cuts each blob into equal slabs, and Ward’s cuts are the closest to the coloured regions of the ground truth, especially for LAYING and WALKING_UPSTAIRS. None of the methods separates the SITTING and STANDING regions, which overlap even in the ground truth plot.

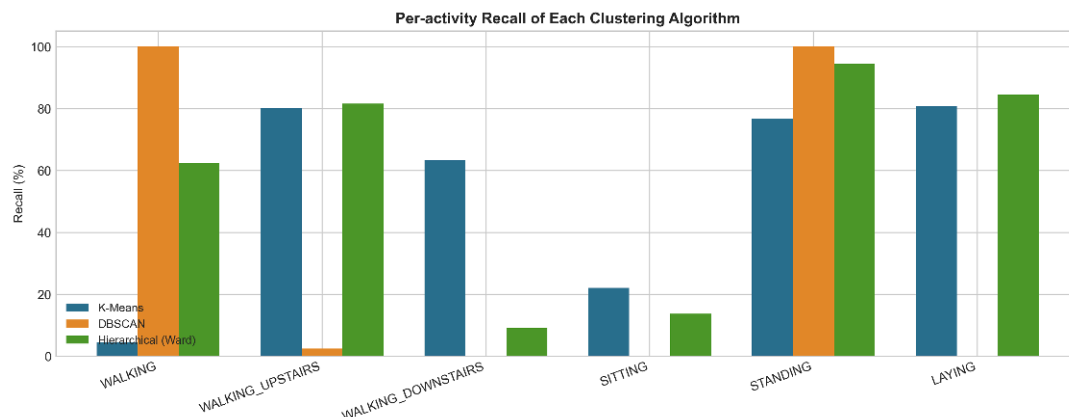


Figure 19: Per-activity recall of each algorithm after cluster-to-activity mapping.

Inference: LAYING, STANDING and WALKING_UPSTAIRS are recovered well by K-Means and Ward (76% to 94% recall), while SITTING (22% and 14%) and WALKING_DOWNSTAIRS (63% and 9%) are mostly merged into their neighbours. These two activities are the least distinctive in the feature space and would need supervised learning to be told apart reliably.

11 Discussion

11.1 Which algorithm produced the most meaningful clusters? Why?

Ward hierarchical clustering came closest to the six activities (ARI 0.490, NMI 0.610, 59.6% mapped accuracy), then K-Means with $k = 6$ (ARI 0.427, NMI 0.562) and DBSCAN (ARI 0.321, NMI 0.500). Ward works like K-Means in spirit, since it also minimises variance, but it builds the partition from the bottom up and so follows the shape of the blobs, which is why it isolates LAYING and WALKING_UPSTAIRS so cleanly. DBSCAN has the best internal scores, but only because it stops at the two dense groups plus one tiny cluster; a high silhouette here does not mean the activities were found. Nothing separates SITTING from STANDING or the three walking variants, because those simply overlap in the feature space.

11.2 How sensitive was K-Means to the choice of k ?

Quite sensitive. The silhouette falls from 0.561 at $k = 2$ to 0.297 at $k = 4$ and then hovers around 0.25 to 0.28, and ARI moves from 0.33 down to 0.29 at $k = 4$ and 5 and then jumps to 0.43 at $k = 6$, so picking 5 instead of 6 costs 0.14 ARI. Initialisation mattered too: with a single random start, 4 of 10 seeds ended in a worse local minimum (WCSS about 888,000 instead of 866,550) with ARI 0.29 instead of 0.43, which is why 10 restarts were used. The feature space hardly mattered (ARI 0.420 in 561-D versus 0.427 in 10-D).

11.3 Did DBSCAN detect noise or small clusters effectively?

Noise, yes. The 830 noise windows (8.1%) are mostly WALKING_DOWNSTAIRS (414, almost a third of that activity) and WALKING (199), the most variable movements, which sit in the sparsest part of the space, so calling them outliers is fair. Small clusters, not really. Besides the two big groups it found one cluster of 34 WALKING_UPSTAIRS windows, and the finer setting ($\epsilon = 5.5$, $\text{minPts} = 10$) only produced fragments of 10 to 39 windows; pushing ϵ lower just turned windows into noise (52.6% at $\epsilon = 2.5$). It also only worked after PCA, since in 561 dimensions there was no density contrast at all.

11.4 How does linkage choice (single/complete/ward) affect hierarchical clustering?

It changes everything. Single linkage chained the data into one cluster of 10,293 windows plus five singletons (ARI 0.000), complete linkage put 9,912 windows in one cluster with a few outlier satellites (ARI 0.004), and average linkage only found the static/dynamic split (ARI 0.330). Ward was the only one that gave balanced clusters (3,344 / 2,399 / 2,113 / 1,717 / 565 / 161) and the best ARI and NMI. Linkages driven by the nearest or farthest pair of points are hostage to a handful of outliers, whereas Ward’s variance rule prefers compact groups of similar size, which is what the activities look like.

11.5 Which internal metric best matched your visual intuition of cluster quality?

Calinski–Harabasz. It punished the degenerate single (32) and complete (379) linkage solutions that look plainly wrong in the t-SNE plots, and scored the solutions that follow the two dense groups highest (16,613 for $k = 2$, 10,399 for DBSCAN). Silhouette and Davies–Bouldin were fooled by degenerate solutions: single linkage got a silhouette of 0.370 and the best Davies–Bouldin of all (0.311), because a singleton is perfectly compact. All internal metrics preferred two clusters while the external ones preferred six, which is the main lesson here: internal metrics measure geometry, and the geometry of this data has two strong groups, not six.

11.6 Strengths, limitations and improvements

K-Means is fast (0.1 s in PCA space) and gave a sensible six-way split but needs k and restarts. DBSCAN needs no k and flags outliers, but its parameters are fiddly, it breaks in high dimensions and cannot split regions of even density. Ward gave the best match and an informative dendrogram at $O(n^2)$ cost, still only 1.1 s here. Things that could help: clustering on features chosen for activity discrimination (gravity and jerk features, say), a Gaussian mixture model that allows elongated clusters, running DBSCAN separately inside the static and dynamic groups with different ϵ , or a few labelled windows as constraints.

12 Conclusion

Three clustering algorithms were applied to 10,299 HAR windows in a standardized 10-component PCA space. The elbow and silhouette analysis showed that the data is dominated by a two-cluster static/dynamic structure (silhouette 0.561 at $k = 2$) with a weaker six-cluster structure underneath (local silhouette peak and best ARI/NMI at $k = 6$), and $k = 6$ was chosen for K-Means. DBSCAN, tuned with a k-distance plot and a grid over ϵ and $minPts$, recovered the two dense groups and flagged 8.1% of the windows as noise but could not split the groups further. Ward hierarchical clustering gave the most meaningful six clusters (ARI 0.490, NMI 0.610, 59.6% mapped accuracy), K-Means was second (ARI 0.427, NMI 0.562) and DBSCAN third (ARI 0.321, NMI 0.500), while single, complete and average linkage produced degenerate partitions. Internal metrics favoured the coarse two-group solutions and were fooled by degenerate ones, with Calinski–Harabasz being the most reliable, so external metrics or domain knowledge remain necessary to judge whether clusters correspond to the activities of interest.

13 References

- [1] D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. L. Reyes-Ortiz, “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” in *Proc. 21st European Symposium on Artificial Neural Networks (ESANN)*, Bruges, Belgium, 2013, pp. 437–442.

- [2] S. P. Lloyd, “Least Squares Quantization in PCM,” *IEEE Transactions on Information Theory*, vol. 28, no. 2, pp. 129–137, 1982.
- [3] D. Arthur and S. Vassilvitskii, “k-means++: The Advantages of Careful Seeding,” in *Proc. 18th ACM-SIAM Symposium on Discrete Algorithms*, 2007, pp. 1027–1035.
- [4] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, “A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise,” in *Proc. 2nd Int. Conf. on Knowledge Discovery and Data Mining (KDD)*, 1996, pp. 226–231.
- [5] J. H. Ward, “Hierarchical Grouping to Optimize an Objective Function,” *Journal of the American Statistical Association*, vol. 58, no. 301, pp. 236–244, 1963.
- [6] P. J. Rousseeuw, “Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis,” *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, 1987.
- [7] D. L. Davies and D. W. Bouldin, “A Cluster Separation Measure,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-1, no. 2, pp. 224–227, 1979.
- [8] T. Caliński and J. Harabasz, “A Dendrite Method for Cluster Analysis,” *Communications in Statistics*, vol. 3, no. 1, pp. 1–27, 1974.
- [9] L. Hubert and P. Arabie, “Comparing Partitions,” *Journal of Classification*, vol. 2, no. 1, pp. 193–218, 1985.
- [10] L. van der Maaten and G. Hinton, “Visualizing Data using t-SNE,” *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008.
- [11] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.